WEEK 1 · DAY 1 · OVERVIEW

Time Value of Money

From your first line of code to your own AI agents.
Every lesson is a fixed income problem.



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 9 SECTIONS / 5 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

From today the desk computes its own prices, in one module you write, with Excel as the answer key. Four new bonds settle today, 30 September 2026. Start at the base: what 100 grows to at Quorvane's yield, and what each bond's 100 at maturity is worth today. Show me, in dollars, how much the compounding convention moves that. Put both functions in your own file, docstring first. Prove they match Excel on every row of the answer key. Save a version you can go back to, and take it home.

ABOUT 50 MIN WITH EXERCISES / 9 SECTIONS / 5 EXERCISES

Your Route Through Day 1

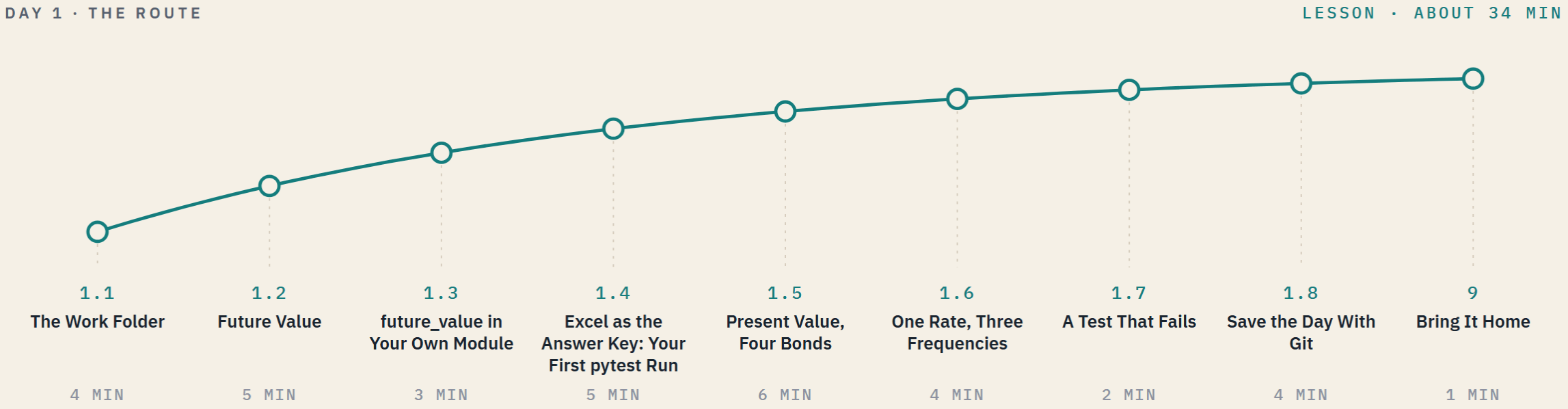


FIGURE 1 Nine sections, in order, with about how long each takes.

Time. About 34 minutes for the lesson, then about 15 for the five exercises.

1.1 The Work Folder

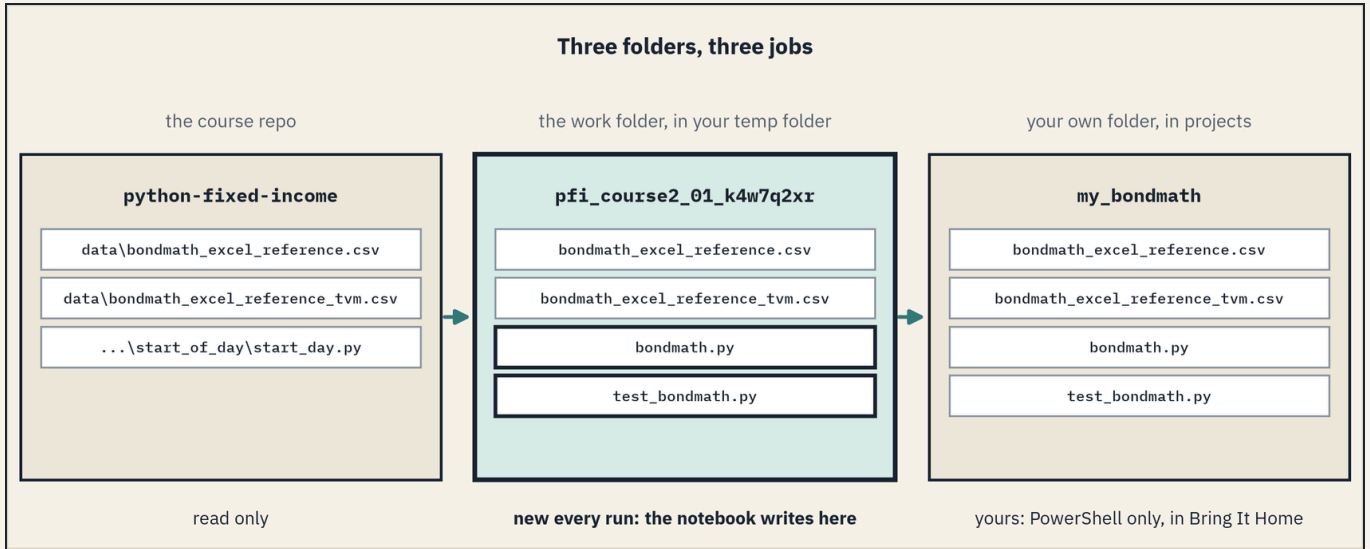


FIGURE 2 Three folders: the read-only repo, a fresh temp work folder, and your own my_bondmath.

EXCEL'S ANSWER, ONE ROW

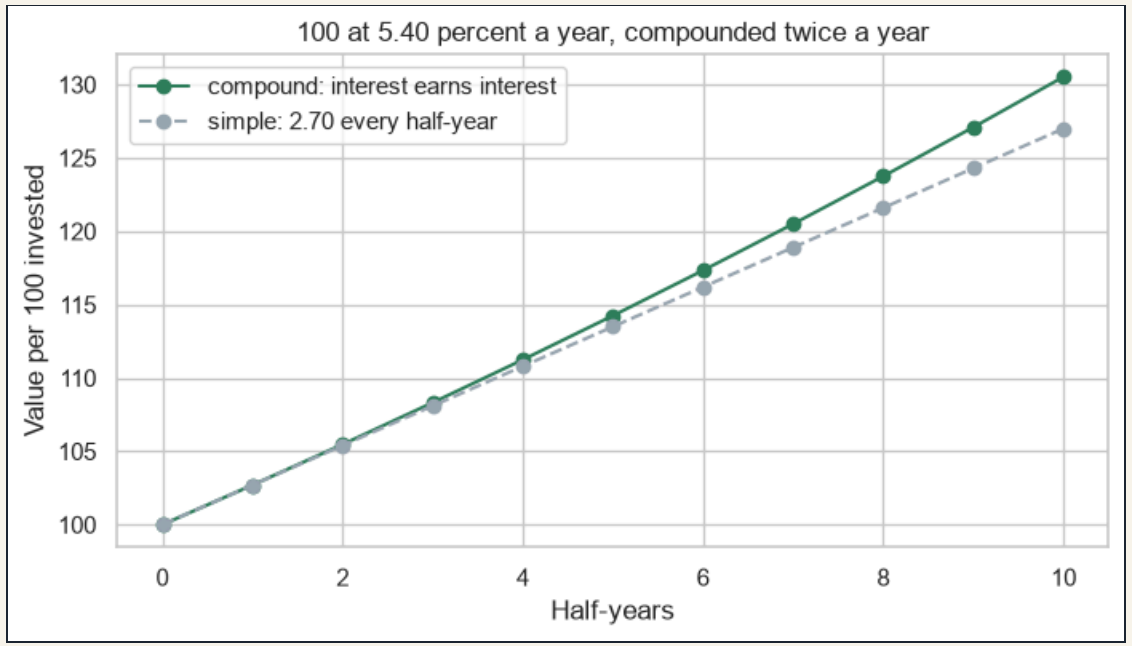
```
153 rows. Row tvm_fv_09:
=FV(5.4/100/2,2*5,0,-100) returned 130.52822611608937
```

TODAY'S FOUR BONDS

	TICKER	COUPON_PCT	YEARS	YIELD_PCT
0	QVNE	5.250	5	5.400
1	DNMR	6.125	7	6.125
2	PLWK	4.750	3	4.600
3	HLVF	7.500	10	7.700

The work folder holds Excel's answers; everything the notebook writes lands beside them.

1.2 Future Value



THE FORMULA

```
growth_factor = (1 + period_rate) ** (2 * 5)
```

```
Growth factor: 1.3053. Future value: 130.5282
```

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
Growth factor	`=(1+5.4/100/2)^(2*5)`	1.3053
Future value	`=FV(5.4/100/2,2*5,0,-100)`	130.5282

One formula grows 100 to Excel's 130.5282: the loop, written once.

1.3 future_value in Your Own Module

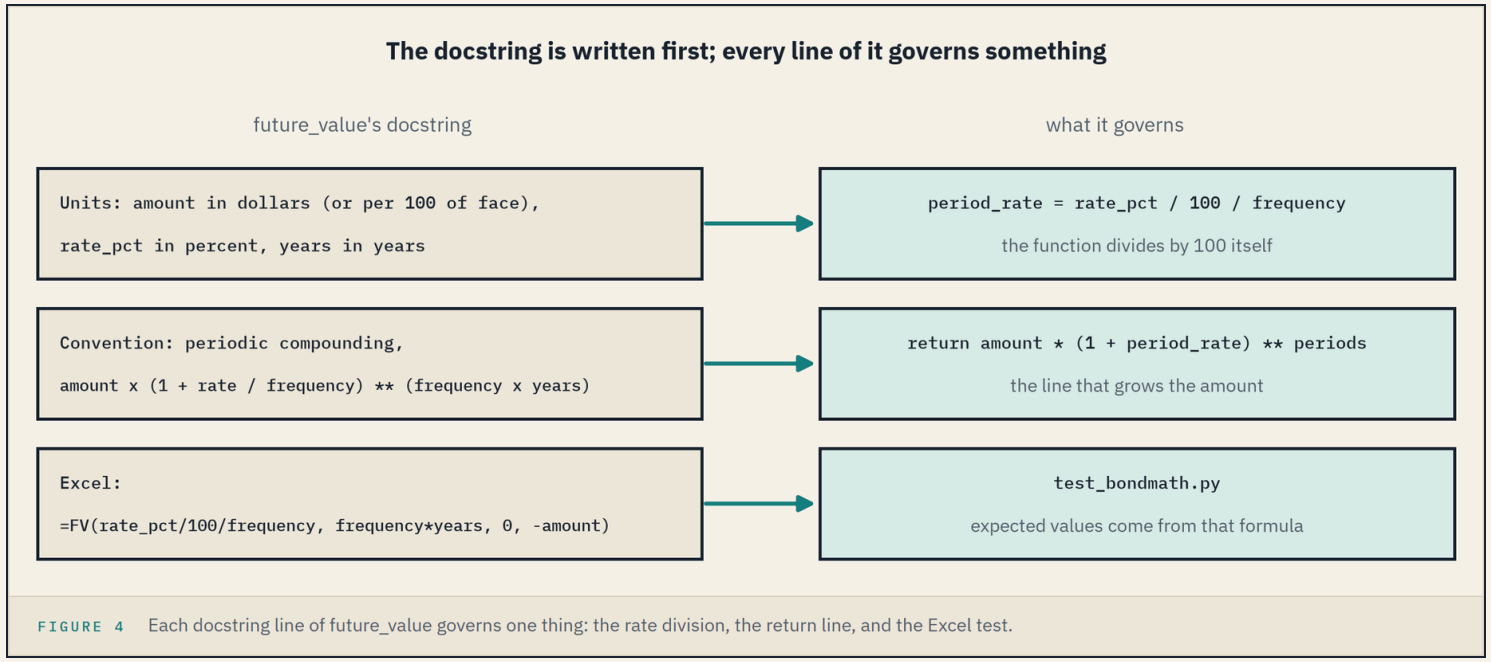


FIGURE 4 Each docstring line of future_value governs one thing: the rate division, the return line, and the Excel test.

IMPORT IT

```
import bondmath
print(bondmath.future_value(100, 5.4, 5))
```

```
130.5282261160894
```

The function lives in a file now, and its docstring says what it promises.

1.4 Excel as the Answer Key: Your First pytest Run

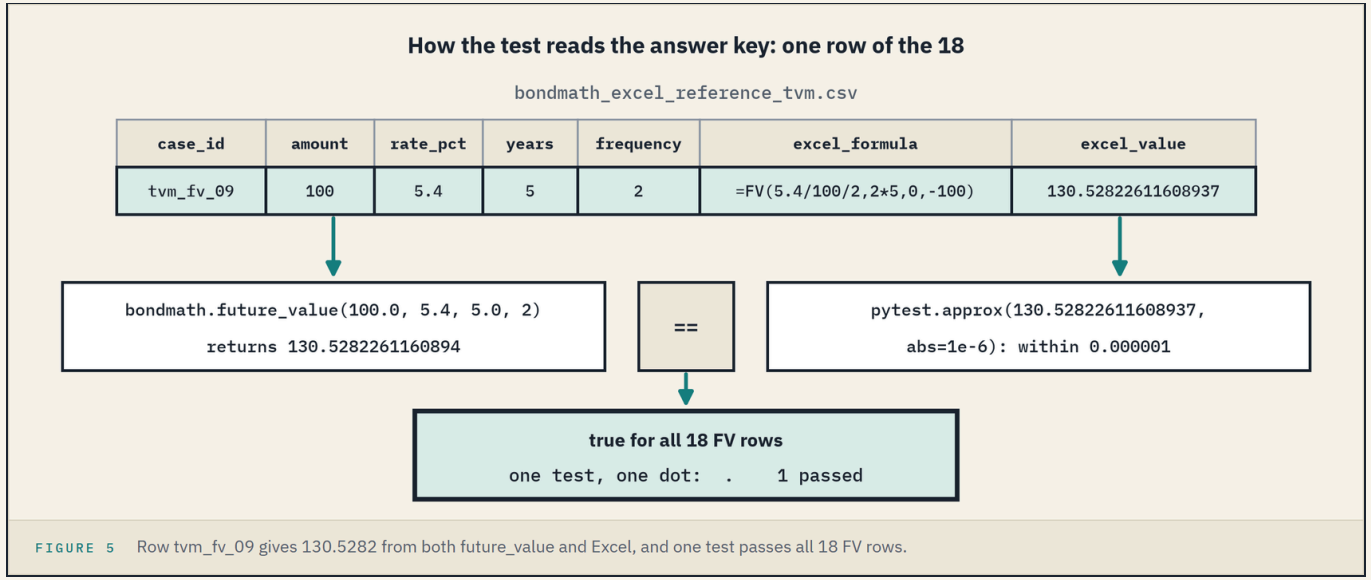


FIGURE 5 Row tvm_fv_09 gives 130.5282 from both future_value and Excel, and one test passes all 18 FV rows.

WHICH ROWS IT READ

```
18 FV rows: 0.5 to 30.0 years, 0.5 to 8.6 percent
Rows by frequency: {1: 2, 2: 13, 4: 3}
```

EXCEL SIDE BY SIDE

A check column does the same job for one row: Python's 130.5282261160894 in A2, `=FV(5.4/100/2,2*5,0,-100)` in B2, and `=ABS(A2-B2)<0.000001` in C2 shows TRUE.

1 passed: **your function agrees with Excel on all 18 FV rows.**

1.5 Present Value, Four Bonds

The same ten steps, backwards: Quorvane's 100 on 30 Sep 2031, worth today




```
discount factor = 1 / 1.3053 = 0.766118
```

100 x 0.766118 = 76.6118: divide by the growth factor once, or by 1.027 ten times

FIGURE 6 Quorvane's 100 due 30 Sep 2031 is worth 76.61 today, after dividing by 1.027 ten times.

EVERY BOND'S 100, TODAY

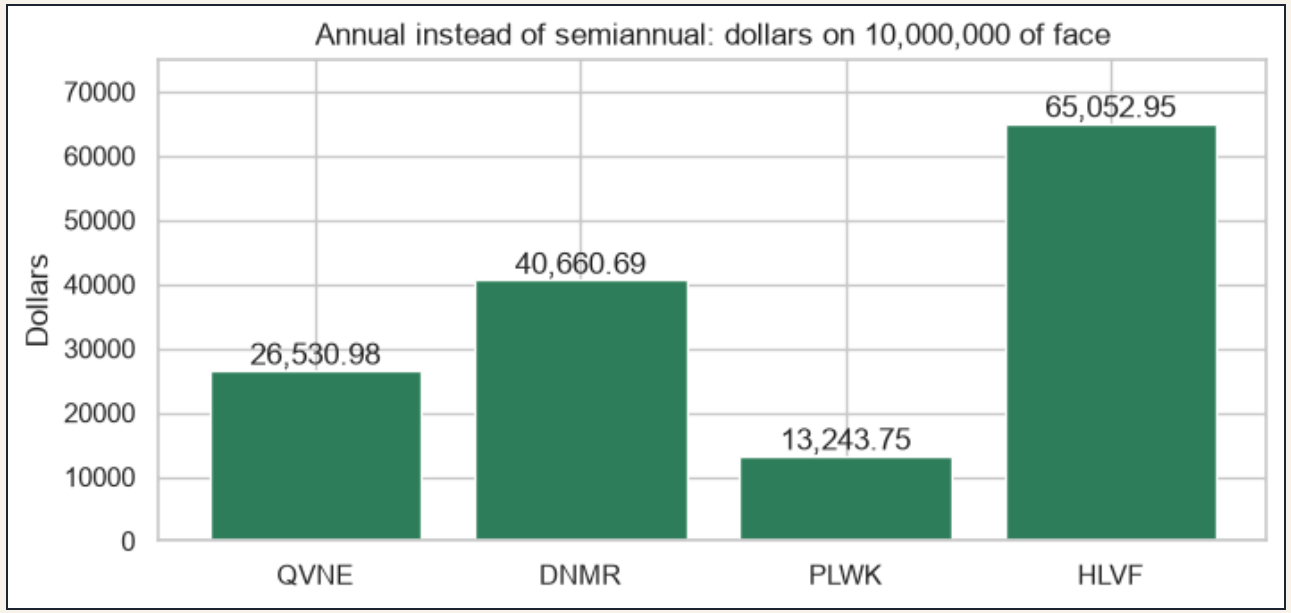
TICKER	YEARS	YIELD_PCT	PV_OF_100
QVNE	5	5.400	76.6118
DNMR	7	6.125	65.5527
PLWK	3	4.600	87.2461
HLVF	10	7.700	46.9753

EXCEL SIDE BY SIDE

EXCEL	RESULT
`=100/(1+5.4/100/2)^(2*5)`	76.6118
`=-PV(5.4/100/2,2*5,0,100)`	76.6118

Discounting is growth run backwards, and `3 passed` covers both directions.

1.6 One Rate, Three Frequencies



ONE RATE, THREE FREQUENCIES

```
1 a year: grows to 130.0778, worth 76.8771 today
2 a year: grows to 130.5282, worth 76.6118 today
4 a year: grows to 130.7600, worth 76.4760 today
```

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
Annual, grown	`=FV(5.4/100/1,1*5,0,-100)`	130.0778
Annual, today	`=PV(5.4/100/1,1*5,0,100)`	-76.8771
Quarterly, grown	`=FV(5.4/100/4,4*5,0,-100)`	130.7600

A yield is not complete without its frequency: the wrong one moves Halvering by 65,052.95 dollars, and nothing warns you.

1.7 A Test That Fails

A COLLEAGUE'S TEST

```
%%writefile test_excel_sign.py
"""A colleague's quick test: Quorvane against Excel's PV as Excel returns it."""
import pandas as pd
import pytest
import bondmath
def test_quorvane_against_excel_pv():
    excel = pd.read_csv('bondmath_excel_reference_tvm.csv', index_col='case_id')
    expected = excel.loc['tvm_pv_09', 'excel_value']
    assert bondmath.present_value(100, 5.4, 5) == pytest.approx(expected, abs=1e-6)
```

READ THE FAILURE

The **>** line is the **`assert`**; the **E** line gives the two numbers, equal apart from the sign.

WHOSE FAULT

The function is right and the test is wrong: **`test_bondmath.py`** already compares with minus Excel's value.

A failing test says two things disagree; deciding which is wrong is your job.

1.8 Save the Day With Git

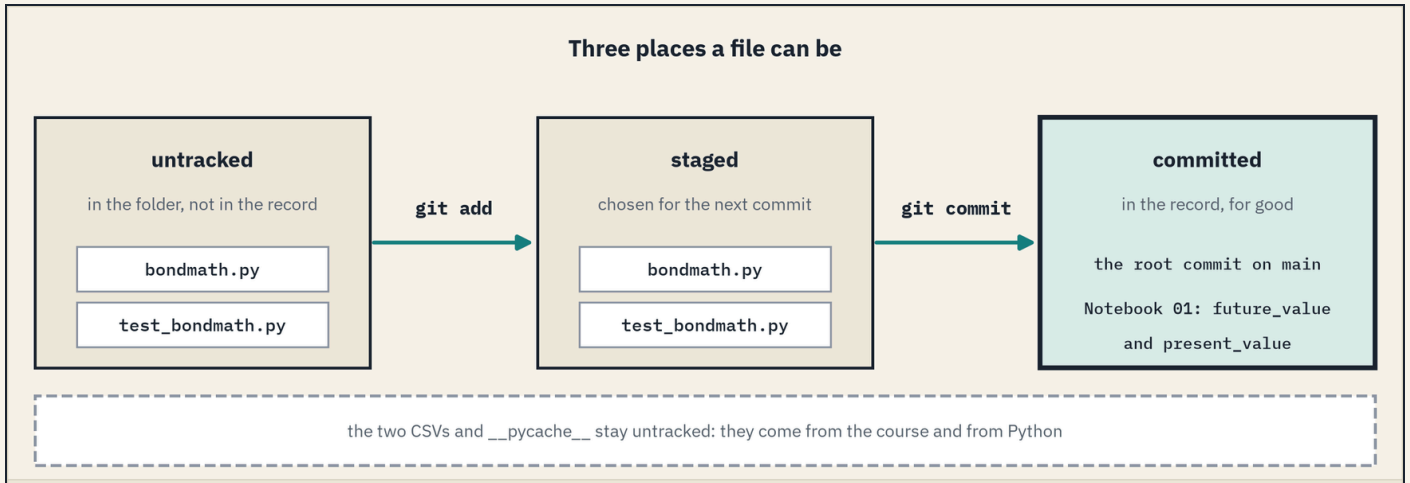


FIGURE 7 A file moves from untracked to staged to committed; the two CSVs and __pycache__ stay untracked.

STAGE THE TWO FILES

```
A  bondmath.py
A  test_bondmath.py
?? __pycache__/
?? bondmath_excel_reference.csv
?? bondmath_excel_reference_tvm.csv
```

EXCEL SIDE BY SIDE

Save As with a date keeps whole copies; a commit keeps each version with a message and the exact lines that changed.

Today's two files are in the record, and every later change can be read line by line.

Bring It Home

IN POWERSHELL

```
# the work folder the setup cell printed
$work = "$env:TEMP\pfi_course2_01_k4w7q2xr"
# make my_bondmath (Out-Null hides the listing) and move into it
New-Item -ItemType Directory -Force "$HOME\projects\my_bondmath" | Out-Null
Set-Location "$HOME\projects\my_bondmath"
# copy in the two files and Excel's answers, then run the tests and commit
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py" .
Copy-Item "$work\bondmath_excel_reference.csv", "$work\bondmath_excel_reference_tvm.csv" .
python -m pytest -q
git init -q -b main
git config user.name "Your Name"
git config user.email "you@example.com"
git add bondmath.py test_bondmath.py
git commit -m "Notebook 01: future_value and present_value"
```

Your own `my_bondmath` holds the module, its tests, and their first commit.

You Can Now

- Grow an amount with one formula: future value.
- Bring a payment back to today: present value.
- Spot a convention error in the frequency.
- Write a function docstring first.
- Test a module against Excel's answers with pytest, and read a pass and a failure.
- Read Excel's **FV** and **PV** with their sign.
- Make a first commit and take the files home.

NEXT
WEEK 1 · DAY 2

Cash Flows and Price From Yield

A bond as a list of payments, its price on a coupon date as their discounted sum, and two more functions for your module, tested against Excel's PV and PRICE.